



STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025 26)
Faculty Name	NM Deepika
Student Name	KOTA MOHITH
Roll Number	2520030520
Branch	CSE
Course Outcome	CO-3

Case Study Topic:GitHub Actions Workflow — Cyclic Dependency Detection using 3-Colour DFS

Work Done Summary:

- Analyzed a GitHub Actions CI/CD workflow automation optimization problem modeling jobs and **needs** constraints as a Directed Graph.
- Explored how a pre-flight validation script identifies accidental deadlocks caused by cyclical dependencies before resources are spun up.
- Studied the 3-color Depth First Search (DFS) state machine optimization tracking vertex lifecycle states as **WHITE**, **GREY**, and **BLACK**.
- Constructed a dependency graph for a 4-job micro-scenario containing a cyclic hazard (**build**, **test**, **integration**) alongside a standalone dependent node (**deploy**).
- Conducted a manual, granular step-by-step trace of the recursive DFS algorithm initialized from the **deploy** job using alphabetical neighbor sorting rules.
- Isolated the exact structural execution step where a back-edge conflict is identified and a cycle exception is thrown.
- Evaluated the mathematical worst-case time complexity of the pre-flight static analyzer.

Implementation Details:

Graph Modeling: Jobs act as vertices (V) and **needs** parameters act as directed edges (E). The edge definition follows dependency flow: if **build** needs **integration**, a directed edge exists from **build** → **integration**.

State Space Augmentation: Each vertex is allocated a color property to track loop history:

- **WHITE (0):** Unvisited node. Default initial state.
- **GREY (1):** Visiting node. The node is on the current recursion stack (ancestor).
- **BLACK (2):** Fully processed node. All its descendants have been completely traversed.

Edge Classification Rules: * Encountering a **GREY** node implies running into an active ancestor still sitting on the execution stack, confirming a **Back-Edge** (Cycle Detected).

- Encountering a **BLACK** node implies running into an already completed branch, classifying it as a **Cross/Forward-Edge** (Safe, No Cycle).

Part (a): Theory, Traversal Trace, and Complexity Analysis

(i) 3-Color DFS Cycle Detection Mechanics

The algorithm maps structural loops by evaluating the color of the target vertex whenever a directed edge (u,v) is traversed:

- **Edge to a GREY Vertex:** This indicates that vertex v is an ancestor of u in the current DFS path. Because v was visited earlier but is not yet closed (still on the active recursion stack), returning to it creates a **back-edge**, confirming a **cyclic dependency**.
- **Edge to a BLACK Vertex:** This indicates that vertex v and all its downstream branches have already been completely traversed and closed in an earlier DFS path. Encountering it creates a forward or cross-edge, which represents a safe, valid reuse of shared dependencies. No cycle is reported.

(ii) Graph Trace from Node **deploy** in Alphabetical Neighbor Order

Graph Configuration & Adjacency List: Based on the scenario text, we map the dependencies out explicitly:

- **deploy** needs **build** → **deploy** → **build**
- **build** needs **test** → **build** → **test**
- **test** needs **integration** → **test** → **integration**
- **integration** needs **build** → **integration** → **build**

Step-by-Step Traversal Trace:

1. **Initialize Engine:** All nodes set to **WHITE** (build:W, deploy:W, integration:W, test:W).
2. **Call dfs(deploy):** Mark **deploy** as **GREY** (Active Stack: [**deploy**]).
3. **Traverse deploy → build:** Node **build** is **WHITE**. Call **dfs(build)**.
4. **Inside dfs(build):** Mark **build** as **GREY** (Active Stack: [**deploy**, **build**]).

5. **Traverse `build` → `test`:** Node `test` is **WHITE**. Call `dfs(test)`.
6. **Inside `dfs(test)`:** Mark `test` as **GREY** (Active Stack: [`deploy`, `build`, `test`]).
7. **Traverse `test` → `integration`:** Node `integration` is **WHITE**. Call `dfs(integration)`.
8. **Inside `dfs(integration)`:** Mark `integration` as **GREY** (Active Stack: [`deploy`, `build`, `test`, `integration`]).
9. **Traverse `integration` → `build`:** Evaluate the color status of target vertex `build`.

CRITICAL DETECTION STEP: At Step 9, vertex `build` is found to be **GREY**. The algorithm identifies this transition as an explicit **back-edge**, immediately raises a "cycle detected" flag, and halts execution before any jobs are run.

(iii) Time Complexity Analysis

- **Time Complexity:** $O(V+E)$ where V is the number of jobs (12 in the main scenario) and E is the number of dependency links.
- **Justification:** In the worst-case scenario where no cycles are present, the algorithm inspects every node exactly once when transitioning its state from **WHITE** to **GREY**, and scans each outgoing edge exactly once to check the adjacent vertex's state. This linear processing runtime allows the CI/CD pre-flight validator to safely reject malformed pull requests in a few milliseconds.

```

Listening on 50082
User program running
User program finished
--- Elasticsearch Multi-Term Query Engine ---
Loading initial posting lists...
list_machine : [1, 4, 7, 12, 15, 20, 24, 30]
list_learning : [2, 4, 8, 12, 18, 24, 30]
list_tutorial : [4, 6, 9, 12, 17, 24, 30, 35]

[Optimization] Order of intersection (shortest first):
List 1 (Size 7): [2, 4, 8, 12, 18, 24, 30]
List 2 (Size 8): [1, 4, 7, 12, 15, 20, 24, 30]
List 3 (Size 8): [4, 6, 9, 12, 17, 24, 30, 35]

Executing Intersection step 1...
-> Intermediate Matches: [4, 12, 24, 30]
Executing Intersection step 2...
-> Intermediate Matches: [4, 12, 24, 30]

-----
FINAL INTERSECTED MATCHES (Document IDs): [4, 12, 24, 30]
Time Complexity (worst case):  $O(|L1| + |L2| + |L3|)$ 
-----
Process finished with exit code 0

```

GitHub Link: https://github.com/kotamohith274/DSA-_2-Case-Studies.

Student Signature	Faculty Signature
_____	_____